Oracle Table Re-org Framework

Stateful, Resumable, Downtime-Safe Database Maintenance

Technical Design & Implementation Document

Version 1.0 | April 2026 | DBA Team

Schema	Table	Steps Executed	Duration	Result
IMS_MM	NACHA_BATCH_REP_DTLS	9,595	~22 mins	100% SUCCESS
RTGS_MT	RTGS_IN_REPOSITORY	7,537	N/A	100% SUCCESS

1. Overview

During planned downtime windows, Oracle databases accumulate fragmented segments in tables, partitions, and LOB columns. The standard remediation — running ALTER TABLE MOVE and ALTER INDEX REBUILD — reclaims this space. However, with tables containing tens of thousands of subpartitions or indexes, a single reorg run can easily exceed the available downtime window.

This framework solves that problem by introducing stateful, resumable table re-organization. Every DDL step is registered in a checkpoint table (REORG_TRACKER) before execution. If the session is killed or time runs out, the next run reads the checkpoint table, skips all DONE steps, and resumes from exactly where it stopped — no work is repeated, no step is missed.

> **Key Capability**
> Kill the session at any point during a 6-hour downtime. Restart the next day. The framework picks up from the exact step that was interrupted. Every completed step is protected.

1.1 Problem Statement

Challenge	Impact	This Framework's Solution
Table reorg exceeds downtime window	Cannot complete in one session	Checkpoint every step — resume next window
Killed session loses all progress	Restart from zero	REORG_TRACKER persists every DONE step
ORA-14287 on composite indexes	Script aborts	Detect and exclude non-rebuildable partitions
10,000+ subpartitions / indexes	Fixed ranges overflow	Continuous seq counter — no range limits
Index rebuild after every subpart	Inconsistent state	Batch-20 pattern: move 20 → rebuild those 20's indexes

1.2 System Components

File	Type	Purpose	Run When
01_create_tracker_table.sql	DDL	Creates REORG_TRACKER checkpoint table	Once only (setup)
02_reorg_procedure.sql	PL/SQL	Core engine — seeds, recovers, executes all DDL	Once to compile
03_run.sql	SQL*Plus Script	Runner — pre-checks, calls procedure, post-checks, HTML output	Every downtime window
04_monitor_and_utilities.sql	SQL Queries	Live progress monitoring and manual control	Any time during run

2. Architecture & Workflow

2.1 High-Level Flow

The framework operates in a clear three-layer model:

Layer	Component	Responsibility
Execution Layer	03_run.sql (SQL*Plus)	Drives the session: pre-checks → calls procedure → post-checks → HTML spool
Logic Layer	REORG_TABLE procedure	Seeds steps, recovers killed sessions, executes DDL in correct order
State Layer	REORG_TRACKER table	Persistent checkpoint — every step's status, timing, and error recorded

2.2 Execution Phases Inside the Procedure

Phase	Name	What Happens
0	Session Settings	ALTER SESSION FORCE PARALLEL QUERY/DDL PARALLEL 16
1	SEED	Inspects dba_tab_partitions, dba_lobs, dba_indexes. Inserts every DDL step as PENDING. Idempotent — existing rows skipped, DONE rows never touched.
2	RECOVER	Finds any step stuck as RUNNING (from a killed session) and resets to PENDING automatically.
3	EXECUTE	Loops PENDING/FAILED steps in step_seq order. Marks RUNNING before DDL, DONE/FAILED after. Applies batch-20 subpartition index rebuild pattern.

2.3 Step Execution Order (Dependency-Safe)

Steps are seeded in this fixed order so that data moves always precede index rebuilds:

Order	Step Type	DDL Issued	Notes
1	MOVE_TABLE	ALTER TABLE ... MOVE PARALLEL 16	Non-partitioned tables only
2	MOVE_PARTITION	ALTER TABLE ... MOVE PARTITION ... PARALLEL 16	Single-level partitioned tables
3	MOVE_SUBPART	ALTER TABLE ... MOVE SUBPARTITION ...	Composite partitioned — batch 20 at a time
4	MOVE_LOB	ALTER TABLE ... MOVE LOB (...) STORE AS (...)	Non-partitioned LOB columns
5	MOVE_LOB_PART	ALTER TABLE ... MOVE PARTITION ... LOB (...)	Partitioned LOB segments
6	REBUILD_IDX_PART	ALTER INDEX ... REBUILD PARTITION ... PARALLEL 16	Single-level partitioned indexes only (ORA-14287 fix)

Order	Step Type	DDL Issued	Notes
7	REBUILD_IDX	ALTER INDEX ... REBUILD PARALLEL 16	Non-partitioned regular indexes
8	REBUILD_IDX_SP	ALTER INDEX ... REBUILD SUBPARTITION ... PARALLEL 16	Composite partitioned index subpartitions
9	REBUILD_LOB_PART	ALTER INDEX ... REBUILD PARTITION ...	Single-level LOB index partitions
10	REBUILD_LOB_IDX	ALTER INDEX ... REBUILD	Non-partitioned LOB indexes

2.4 Resume Logic — How It Works

The resume capability is built on three simple guarantees:

- COMMIT happens immediately after marking a step DONE — so even a kill 1 second later preserves the completed work
- Before each DDL fires, the step is marked RUNNING + COMMITted — so a killed session leaves a visible breadcrumb
- On next run, Phase 2 detects any RUNNING row and resets it to PENDING so it re-runs cleanly

Resume Example
Day 1: Run starts at 9 PM, kills at 3 AM. Steps 1-4000 = DONE, step 4001 = RUNNING (killed mid-DDL). Day 2: Same command. Phase 2 resets step 4001 → PENDING. Execute loop skips steps 1-4000 (DONE), starts at step 4001.

3. Batch-20 Subpartition Pattern

This is the most technically interesting part of the framework. When a table has a composite partition structure, moving a subpartition renders the corresponding index subpartition UNUSABLE. The original script handled this by moving 20 subpartitions in a BULK COLLECT batch, then rebuilding all UNUSABLE index subpartitions within that batch before moving on.

The framework preserves this exact behavior:

Step	What Happens
Move SUBPART_001 → SUBPART_020	Each name is added to v_batch_names array. Counter reaches 20.
Batch threshold hit (count = 20)	rebuild_batch_subpart_indexes() is called.
Inside rebuild procedure	For each of the 20 names: query dba_ind_subpartitions WHERE subpartition_name = that name AND status = UNUSABLE. Rebuild only those.
Batch cleared	v_batch_names deleted, counter reset to 0.
Move SUBPART_021 → SUBPART_040	Next batch begins — same cycle.
Final remainder (< 20)	Flushed when first non-subpart step is encountered, or at end of loop.

Why Filter by Name?
When you MOVE SUBPARTITION MF_ACT_20250901, the index subpartition named MF_ACT_20250901 becomes UNUSABLE. The rebuild query filters WHERE subpartition_name = 'MF_ACT_20250901'. This means only the indexes related to that exact subpartition are rebuilt — not the whole table's indexes. This is both correct and efficient.

3.1 Real Execution Proof (from Alert Log)

Log Entry	Meaning
RUN [1] MOVE_SUBPART → MF_ACT_20250831	First subpartition moved
DONE [1] 3s	Completed in 3 seconds, name added to batch list [1]
... steps 2-20 ...	Next 19 subpartitions moved
DONE [20] 0.6s	Batch counter = 20, threshold hit
Batch of 20 done → rebuild index subparts for this batch	rebuild_batch_subpart_indexes() called for names 1-20 only
RUN [21] MOVE_SUBPART → MF_ACT_20250920	Next batch starts fresh
Flush final batch (16 subparts) → rebuild their index subparts	Remainder flushed at end (436 total: 21×20=420 + 16 remainder)

4. Script-by-Script Breakdown

4.1 01_create_tracker_table.sql

Run once as DBA. Creates the REORG_TRACKER table which is the entire state management backbone of the framework.

Column	Type	Purpose
schema_name	VARCHAR2(30)	Target schema
table_name	VARCHAR2(128)	Target table
step_seq	NUMBER	Execution order — lower numbers run first (continuous, no gaps)
step_type	VARCHAR2(30)	DDL category: MOVE_TABLE, MOVE_SUBPART, REBUILD_IDX_SP etc.
step_target	VARCHAR2(128)	Partition or index name
step_target2	VARCHAR2(128)	LOB column name (if applicable)
ddl_statement	VARCHAR2(4000)	Exact DDL stored verbatim — no reconstruction needed at runtime
status	VARCHAR2(10)	PENDING / RUNNING / DONE / FAILED
started_at / completed_at	TIMESTAMP	Precise timing per step
elapsed_secs	NUMBER	How long the DDL took
error_msg	VARCHAR2(4000)	ORA- error captured on failure

4.2 02_reorg_procedure.sql — Core Engine

The main PL/SQL procedure. Contains four sub-procedures and the main execution block.

Sub-Procedure	Purpose
log_alert()	Writes timestamped progress messages to the DB alert log via DBMS_SYSTEM.ksdwrt. Visible in real-time even if the session is running silently.
add_step()	Inserts one PENDING row into REORG_TRACKER. Uses INSERT ... WHERE NOT EXISTS so it is completely safe to call on resume — existing rows are silently skipped, DONE rows are never overwritten.
rebuild_batch_subpart_indexes()	Loops through the v_batch_names array (up to 20 names), queries dba_ind_subpartitions for UNUSABLE rows matching each name, and issues ALTER INDEX REBUILD SUBPARTITION. Clears the array after completion.

4.3 03_run.sql — The Daily Driver

This is the only script you run each downtime. Edit 2 lines (schema + table name), run it, walk away.

Section	What It Does
SQL*Plus Settings	SET MARKUP HTML ON ENTMAP OFF — enables SQL*Plus default HTML table output without custom CSS. TRIMSPOOL, LINESIZE, PAGESIZE configured.
SPOOL open	Opens SCHEMA_TABLE.html in the current directory. No UTL_FILE, no directory objects required.
PRE-REORG Queries	Runs all 7 original diagnostic queries: object counts, total size, partition-wise size, index status (non-part, partition, subpartition, summary, LOB indexes, LOB index partitions).
SET MARKUP HTML OFF	Turns off HTML wrapping before EXEC so procedure output does not appear as raw HTML text in the file.
EXEC REORG_TABLE()	Calls the procedure — does all the work. Progress visible in alert log.
SET MARKUP HTML ON	Turns HTML back on for post-check queries.
REORG_TRACKER queries	Shows step-by-step results: SEQ, TYPE, TARGET, STATUS, SECS, STARTED, COMPLETED, ERROR.
POST-REORG Queries	Exact same 7 queries repeated — compare PRE vs POST in one HTML file.
SPOOL OFF	Closes and saves the HTML file.

4.4 04_monitor_and_utilities.sql — Live Control

Run these queries in a separate SQL Developer session while the reorg is running.

Query / Utility	Use Case
Progress for one table	Shows every step with status, elapsed time, and any error message
Summary across all tables	DONE/PENDING/FAILED/TOTAL counts and % complete for every table in tracker
Reset FAILED steps	After fixing root cause, marks failed steps PENDING so they retry next run
Fix RUNNING step manually	For edge cases where auto-recovery did not fire — manually resets stuck RUNNING row
Skip a specific step	Marks one step as SKIPPED if you need to bypass it intentionally
Clean up after completion	DELETE tracker rows once reorg is fully verified and done

5. Execution Results — NACHA_BATCH_REP_DTLS

5.1 Run Summary

Metric	Value
Schema / Table	IMS_MM / NACHA_BATCH_REP_DTLS
Total Steps Seeded	9,596
Total Steps Completed (DONE)	9,595 (100%)
FAILED Steps	0
SP_INDX WARN	0
Index Subparts Rebuilt (via batch logic)	168
Start Time	2026-04-13 21:18:28
End Time	2026-04-13 21:40:49
Total Duration	~22 minutes
Stop & Resume Tested	Yes — confirmed working (screenshot evidence)
Table Structure	3 partitions, 436 subpartitions, 0 LOBs, 15 LOB partitions

5.2 Step Type Breakdown

Step Type	Count	Status	Notes
MOVE_SUBPART	436	All DONE	436 subpartitions moved in 21 batches of 20 + 1 remainder of 16
REBUILD_IDX_SP	9,157	All DONE	Composite index subpartitions rebuilt
REBUILD_IDX	3	All DONE	Non-partitioned regular indexes
SP_INDX (batch inline)	168	All rebuilt	Index subparts rebuilt inline during batch-20 moves

5.3 Batch-20 Pattern in Numbers

Metric	Value
Total subpartitions	436
Full batches of 20	21
Final partial batch	16 subpartitions
Batch trigger messages in log	21 × "Batch of 20 done"
Flush trigger messages in log	1 × "Flush final batch (16 subparts)"
Average batch rebuild time	~15 seconds (from log timestamps)

5.4 Stop & Resume Evidence

The screenshots confirm the framework was paused and restarted during the run:

Screenshot	Observation	Meaning
Screenshot 1	RUNNING: step 6809, REBUILD_IDX_SP, MF_ARC_20260206 at 9:34	Session is actively running mid-step
Screenshot 2	71% complete: DONE=6808, PENDING=2786, STUCK_RUNNING=1	First run in progress — one step RUNNING
Screenshot 3	RUNNING: step 6849, REBUILD_IDX_SP, MF_LIV_20251004 at 9:37	A few minutes later — new step now running
Screenshot 4	75% complete: DONE=7183, PENDING=2411, STUCK_RUNNING=1	Progress confirmed — more steps completing
Screenshot 5 (Alert Log)	RECOVERED 1 RUNNING steps → PENDING. RUN [6809] continues.	After restart — auto-recovery fired, resumed from step 6809
Screenshot 6	DONE=9595, PENDING=0, FAILED=0, PCT=100%	Final state — complete success on both tables

RTGS_MT Result
RTGS_MT.RTGS_IN_REPOSITORY also completed 100% (7,537/7,537 steps DONE, 0 FAILED) as seen in Screenshot 6. Both tables confirmed clean.

6. Step-by-Step Execution Guide

6.1 One-Time Setup (DBA — Run Once)

Step 1: Create the tracker table

```
sqlplus / as sysdba @01_create_tracker_table.sql
```

Step 2: Compile the procedure

```
sqlplus dba_admin/password @02_reorg_procedure.sql
```

6.2 Each Downtime Window

Step 1: Edit 03_run.sql — change only these 2 lines:

```
DEFINE v_schema = 'IMS_MM'
DEFINE v_table  = 'NACHA_BATCH_REP_DTLS'
```

Step 2: Also update the SPOOL filename to match:

```
SPOOL IMS_MM_NACHA_BATCH_REP_DTLS.html
```

Step 3: Run it

```
sqlplus dba_admin/password @03_run.sql
```

Step 4: Monitor in a second session (optional but recommended):

```
-- Live progress
SELECT step_seq, step_type, step_target, status, elapsed_secs
FROM SYS.REORG_TRACKER
WHERE schema_name='IMS_MM' AND table_name='NACHA_BATCH_REP_DTLS'
ORDER BY step_seq;
```

Step 5: Kill session at end of downtime window (Ctrl+C or kill from OS)

Step 6: Next downtime — run the EXACT same command as Step 3. It resumes automatically.

6.3 Confirming Completion

```
SELECT schema_name, table_name, done, pending, failed, pct_done
FROM (SELECT schema_name, table_name,
      SUM(CASE WHEN status='DONE' THEN 1 ELSE 0 END) done,
      SUM(CASE WHEN status='PENDING' THEN 1 ELSE 0 END) pending,
      SUM(CASE WHEN status='FAILED' THEN 1 ELSE 0 END) failed,
      ROUND(SUM(CASE WHEN status='DONE' THEN 1 ELSE 0 END)/COUNT(*)*100)||'%' pct_done
      FROM SYS.REORG_TRACKER GROUP BY schema_name, table_name);
```

Success Criteria
DONE = TOTAL, PENDING = 0, FAILED = 0, PCT_DONE = 100%

7. Key Insights & Design Strengths

7.1 What This Framework Does Well

Strength	Detail
Zero data loss on kill	COMMIT after every single step. Kill at any moment — no completed work is lost.
Idempotent seeding	add_step() uses INSERT WHERE NOT EXISTS. Calling it 100 times is identical to calling it once.
No fixed seq ranges	Continuous counter from 1. Works for any number of subpartitions or indexes — no overflow.
ORA-14287 prevention	dba_part_indexes.subpartitioning_type = NONE filter prevents REBUILD PARTITION on composite indexes.
Batch-20 precision	Index subparts are rebuilt only for the exact 20 subpartition names just moved — not the full table.
Auto-recovery	RUNNING rows from killed sessions reset automatically on next run — no manual intervention needed.
No UTL_FILE / no directory	HTML output via SQL*Plus SPOOL — works on any server, any directory, zero setup.
Alert log visibility	Real-time progress in alert log even when running silently (no DBMS_OUTPUT buffering issue).
Multi-table support	REORG_TRACKER holds rows for any number of schemas and tables simultaneously.

7.2 Possible Enhancements

Enhancement	Benefit
Add a time-limit parameter to procedure	Auto-stop at N hours instead of relying on manual kill
Email notification on completion / failure	Proactive alerting without monitoring sessions
Store HTML report path in REORG_TRACKER run log	Full audit trail of which HTML corresponds to which run
Add LOB subpartition move support	Some tables have LOB subpartitions not covered by current MOVE_LOB_PART logic
Parallel batch execution	Run multiple schema/table combinations in parallel using DBMS_SCHEDULER jobs

7.3 Edge Cases Handled

- Composite partitioned table (has subpartitions) — correctly routes to MOVE_SUBPART, not MOVE_PARTITION
- Non-partitioned table — MOVE_TABLE path, no partition loops executed
- Mixed: some indexes partitioned, some not — both REBUILD_IDX and REBUILD_IDX_PART seeded correctly

- LOB columns: partitioned vs non-partitioned — handled by separate MOVE_LOB and MOVE_LOB_PART steps
- Composite index ORA-14287 — detected at seed time via dba_part_indexes.subpartitioning_type, routed to REBUILD_IDX_SP instead
- Partial final batch (< 20 subpartitions) — flushed at phase transition or end of loop
- Resumed run where batch state is lost — batch starts fresh from 0 (index rebuild may re-run for some names, which is safe)

8. Alert Log Message Reference

All messages are prefixed with [REORG][SCHEMA.TABLE] for easy grep filtering.

Message Pattern	When It Appears	Meaning
START	Procedure begins	Session has started
Structure: parts=N subparts=N lobs=N lob_parts=N	After COUNT queries	Table structure detected
SEED DONE - N steps registered	After seeding	All steps registered in tracker
RECOVERED N RUNNING steps → PENDING	If prev session was killed	Auto-recovery fired — safe to continue
RUN [N] STEP_TYPE → TARGET	Before each DDL	Step about to execute
DONE [N] Xs	After each DDL succeeds	Step completed in X seconds
Batch of 20 done → rebuild index subparts for this batch	Every 20 subpart moves	Batch threshold hit — inline index rebuild firing
SP_INDX: INDEX_NAME/SUBPART_NAME	Inside batch rebuild	One index subpartition rebuilt
SP_INDX WARN: ...	Inside batch rebuild	Non-fatal index rebuild warning
Flush final batch (N subparts) → rebuild their index subparts	At phase transition	Remainder batch flushed
FAILED [N] ORA-XXXXX	After DDL fails	Step failed — logged, continues to next step
ALL STEPS PROCESSED	End of execute loop	All PENDING/FAILED steps processed in this run
FATAL ERROR: ...	Unrecoverable error	Outer exception handler fired — investigate immediately

Grep command to monitor in real-time:

```
tail -f $ORACLE_BASE/diag/rdbms/<db>/<inst>/trace/alert_<inst>.log | grep REORG
```